

A Volleyball Activity Analysis System

Object detection, trajectory prediction and natural-language scene understanding

Carlos Vicente · Javier Revuelta · Ángel Moliné · Jorge Molia

Neural Networks / Deep Learning

Universidad San Jorge

30 May 2026

Contents

1. Project Overview	3
2. System Pipeline (processing order)	3
3. Module Descriptions	4
3.1 Player detector (YOLOv8)	4
3.2 Ball detector (YOLOv8)	4
3.3 Role classifier (CNN)	4
3.4 Trajectory prediction (LSTM).....	5
3.5 Team classifier (MLP).....	6
3.6 Vision-language assistant (Gemma 4)	6
3.7 Integration and interface.....	6
4. Datasets and Source Notebooks	7
5. How to Run	7
6. Design Decisions and Justification	8
7. Conclusions and Lessons Learned	8
Individual contributions	8

1. Project Overview

The goal of this project was to build a single application that analyses volleyball and combines, in one pipeline, the main neural network families covered during the course. We chose volleyball because it is fast, the players are clearly separated into two teams, and it is one of the sports allowed by the assignment (football 11 and basketball were excluded).

The system takes an image, a video file and returns an annotated frame with the players detected and tracked, their team and on-court role, the predicted short-term trajectory of each player, the ball position, referees, and a 2D top-view minimap of the court. On top of that, a vision-language assistant can answer questions about the scene in natural language. Everything is driven from a unified graphical interface, so no part of the system is run from the terminal.

Five trained components are involved: a YOLOv8 detector for the players, a second YOLOv8 detector for the ball, a convolutional network that classifies each player's role, a recurrent network that predicts player trajectories, and a small fully-connected network that assigns players to a team from shirt colour. A pretrained vision-language model (Gemma 4) provides the language layer. The recurrent and fully-connected models, together with the ball detector, were trained on our own match video (a France vs. Japan game), which keeps the training data in the same domain as the footage analysed at inference time.

Summary of trained components

Model	Key metric	Value	Params	Epochs
Player detector (YOLOv8s)	mAP@50	0.834	11.1 M	100
Ball detector (YOLOv8s)	mAP@50	0.942	11.1 M	80
Role classifier (CNN)	Val accuracy	93.6%	0.42 M	25
Trajectory model (LSTM)	Val MSE	0.00105	0.55 M	50
Team classifier (MLP)	Val accuracy	90.8%	1,123	40

2. System Pipeline (processing order)

For every frame the system runs the modules in a fixed order. Each step feeds the next, which is why the order matters:

1. Detect players with the YOLOv8 person detector.
2. Correct the homography for camera motion
3. Discard detections that fall outside the court polygon.
4. Assign persistent IDs across frames with ByteTrack.
5. Decide each player's team (homography side of the net, then overridden by the colour MLP when confident) and role (CNN on the player crop).
6. Update the LSTM trajectory prediction for each tracked player.
7. Detect the ball with the dedicated YOLOv8 ball detector and update its trajectory buffer.
8. Draw boxes, labels, trajectory arrows and the 2D minimap on the output frame.

3. Module Descriptions

3.1 Player detector (YOLOv8)

For the players we fine-tuned a YOLOv8s model (about 11.1 M parameters, 28.4 GFLOPs) on the public Roboflow dataset player.yolov8, which already labels players by team. When the dataset shipped without a validation split we built one with an 80/20 random partition and a fixed seed. Training ran for 100 epochs at 640 px input, batch size 16, with early stopping (patience 20). Because the dataset is not large we augmented it twice: an offline pass with Albumentations (horizontal flips, brightness and hue jitter, small affine transforms; three extra copies per image) and the standard on-the-fly YOLO augmentation, including mosaic and a small amount of mixup.

On the validation set (10 images, 54 player instances) the model reached precision 0.705, recall 0.741, mAP@50 of 0.834 and mAP@50-95 of 0.592. The mAP@50 is solid for a small dataset; the drop at the stricter mAP@50-95 is expected, since tight box localisation is the part that suffers most when training data is limited.

```
Validating /content/runs/detect/runs/detect/volleyball_persons_custom/weights/best.pt...
Ultralytics 8.4.56 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (NVIDIA A100-SXM4-40GB, 40441MiB)
Model summary (fused): 73 layers, 11,125,971 parameters, 0 gradients, 28.4 GFLOPs
<  Class      Images  Instances  Box(P)      R      mAP50  mAP50-95): 100% 1/1 18.5it/s 0.1s
    all         10         54      0.705      0.741    0.834    0.592
Speed: 0.1ms preprocess, 0.9ms inference, 0.0ms loss, 1.3ms postprocess per image
Results saved to /content/runs/detect/runs/detect/volleyball_persons_custom
```

Figure 1. Player detector validation output (Ultralytics).

3.2 Ball detector (YOLOv8)

The ball was the hardest object in the project. It is small, fast and often motion-blurred, and the off-the-shelf COCO “sports ball” detector missed it almost completely on our footage. We therefore built our own dataset, volleyball-ball-a-jpn.yolov8, by extracting frames from our match video and manually annotating the ball. We then fine-tuned a separate YOLOv8s model for 80 epochs at 640 px, batch size 16, patience 30. We reduced the rotation augmentation and disabled mixup, since a ball has no meaningful orientation and mixing two frames hurts a single small target more than it helps.

On its validation set (6 images, 6 ball instances) the detector scored precision 0.977, recall 0.833, mAP@50 of 0.942 and mAP@50-95 of 0.324. The high mAP@50 confirms the detector finds the ball reliably; the low mAP@50-95 reflects loose box fit and a validation set that is too small to read the strict metric with confidence. Even so, on real video the dedicated detector is far better than the COCO baseline, which was the whole point of building the dataset.

```
Validating /content/runs/detect/volleyball_ball_custom/weights/best.pt...
Ultralytics 8.4.56 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (NVIDIA A100-SXM4-40GB, 40441MiB)
Model summary (fused): 73 layers, 11,125,971 parameters, 0 gradients, 28.4 GFLOPs
  Class      Images  Instances  Box(P)      R      mAP50  mAP50-95): 100% 1/1 32.1it/s 0.0s
    all         6         6      0.977      0.833    0.942    0.324
Speed: 0.1ms preprocess, 1.5ms inference, 0.0ms loss, 0.0ms postprocess per image
```

Figure 2. Ball detector validation output (Ultralytics).

3.3 Role classifier (CNN)

Each detected player crop is passed through a custom convolutional network, VolleyballCNN (about 0.42 M parameters). It has four convolutional blocks (Conv–BatchNorm–ReLU–MaxPool, widening from 32 to 256 channels), followed by adaptive average pooling and a fully-connected head with dropout. It classifies the player into one of five roles: attacker, receiver, server, setter and referee. We trained it on player crops cut from the annotated volley_detection (v5) dataset, resized to 224×224,

with flips, colour jitter and small rotations as augmentation, using AdamW (learning rate 1e-3, weight decay 1e-4), a cosine learning-rate schedule and cross-entropy loss. The model reached a validation accuracy of about 93.6%. The same training routine is also exposed inside the GUI, so the classifier can be retrained with different hyperparameters without leaving the application.

3.4 Trajectory prediction (LSTM)

Player movement is modelled by a two-layer bidirectional LSTM (hidden size 128, about 0.55 M parameters). It reads a window of 20 past frames, each described by five values (normalised position x, y, velocity vx, vy and detection confidence), and predicts the next 10 steps.

Two design choices are worth explaining. First, the network predicts displacement relative to the last observed position rather than absolute coordinates. This matters for evaluation: with a displacement target, the naive “stay put” prediction is simply zero, so the model only looks good if it actually learns motion. Second, we split each trajectory temporally (the first 80% of a track trains, the last 20% validates) instead of splitting windows at random, which would leak almost-identical overlapping windows into both sets.

We extracted the trajectories from our match video with the player detector plus ByteTrack, keeping only tracks of at least 30 frames, and sub-sampled one frame in four so there is enough motion between steps. This produced 468 training windows and 56 validation windows. Training used MSE loss, Adam (learning rate 1e-3), a step learning-rate schedule and 50 epochs. The best validation MSE was 0.00105, against a naive-baseline MSE of 0.00116, so the LSTM beats “stay put” by about 10%. The margin is modest because volleyball players do not move very far inside a short window, but on the high-movement examples the predicted path clearly follows the real one.

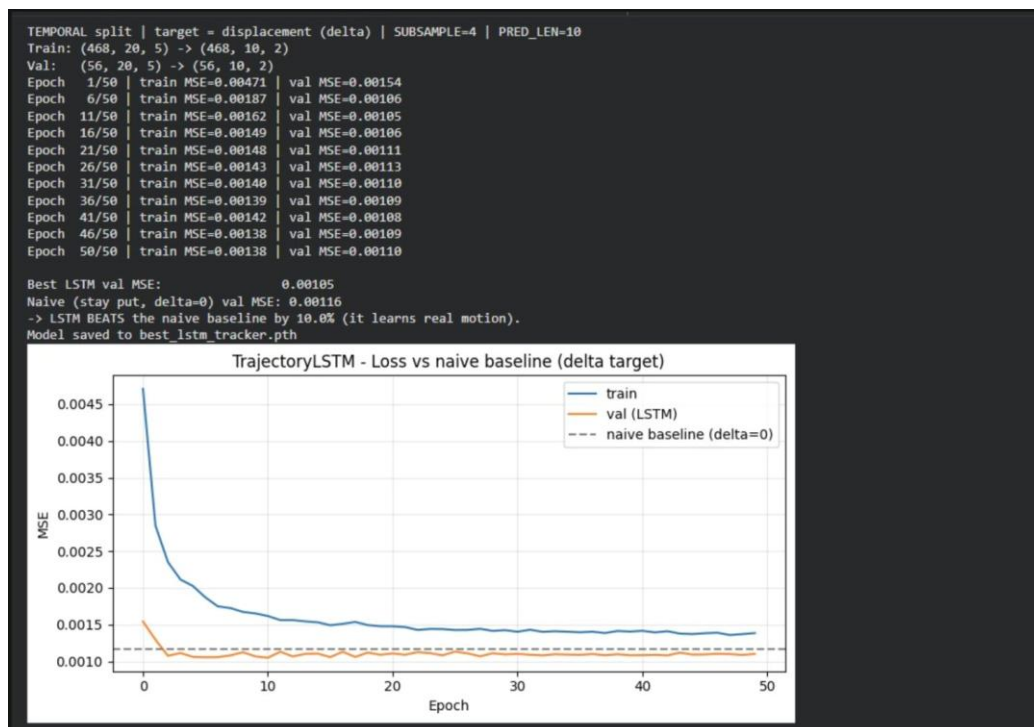


Figure 3. LSTM training log and loss against the naive baseline.

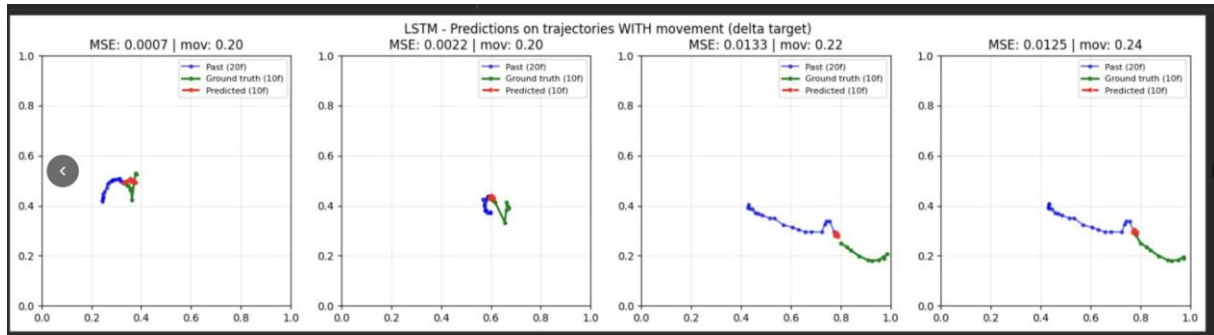


Figure 4. LSTM predictions on the highest-movement validation trajectories (blue: past, green: ground truth, red: predicted).

3.5 Team classifier (MLP)

Assigning a player to team 1, team 2 or referee is done with a small fully-connected network (16→32→16→3, only 1,123 parameters). The input is a 16-bin HSV colour histogram of the player's torso. Rather than label the data by hand, we ran K-means with $k=2$ over the torso histograms of all detected players to discover the two team colours automatically, and marked players standing inside fixed referee zones as the third class. This produced 764 labelled samples (210 / 493 / 61), which is clearly imbalanced, so we trained with class weights ([1.21, 0.52, 4.17]) using cross-entropy, Adam and 40 epochs. The best validation accuracy was 0.908.

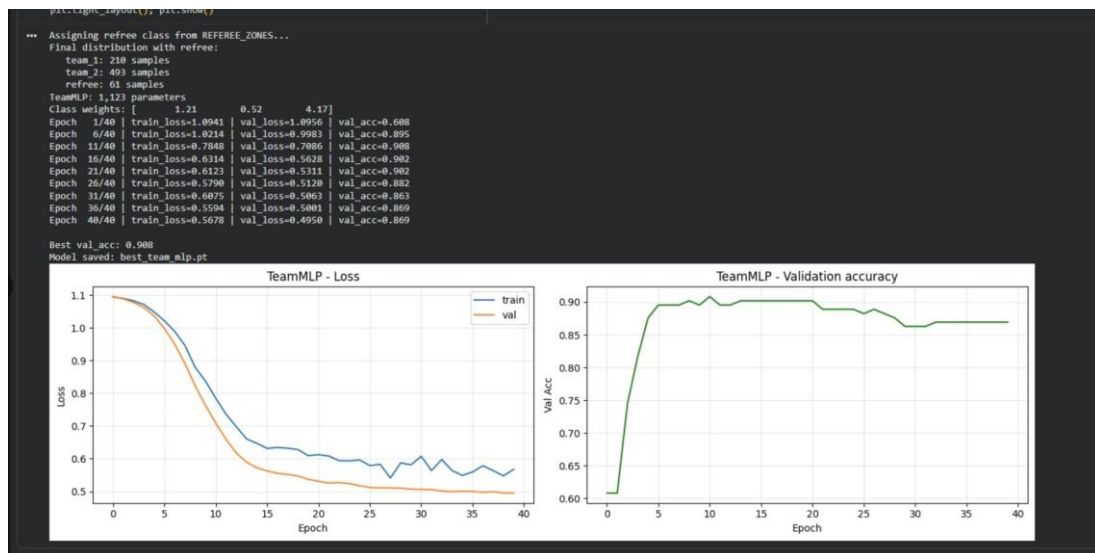


Figure 5. Team MLP training log, loss curves and validation accuracy.

3.6 Vision-language assistant (Gemma 4)

The language layer uses Gemma 4 (unsloth/gemma-4-E2B-it) loaded through Unsloth, which keeps the memory footprint low enough to run alongside the rest of the pipeline. We did not fine-tune it. Instead, the pipeline first analyses the frame, then builds a short structured summary of what it found (players per team, referees, ball position, per-player role and side) and sends that summary together with the annotated image to the model. This way the assistant answers questions such as “how many players from each team?” or “who is closer to the net?” using both the picture and the system's own detections, rather than guessing from the image alone.

3.7 Integration and interface

The components are wired together in a single pipeline class and exposed through a Gradio interface with four tabs: image, video, assistant Q&A, and CNN training. The user never touches a script. The 2D minimap is obtained by projecting player foot positions through the homography, and the homography is corrected per frame so the projection stays aligned when the broadcast camera pans.

4. Datasets and Source Notebooks

Each model was trained in its own notebook and the integrated application lives in a separate one. The datasets used were:

- `player.yolov8` — public Roboflow dataset of volleyball players labelled by team. Used for the player detector.
- `volleyball-ball-a-jpn.yolov8` — dataset we built ourselves by extracting frames from our match video and manually annotating the ball. Used for the ball detector.
- `volley_detection (v5)` — annotated volleyball dataset from which player crops were cut to train the role CNN.
- Match video (France vs. Japan) — the source for the LSTM trajectories and the K-means / MLP team-colour labels.

The source notebooks are:

- `TrainPersonVF.ipynb` — trains the player detector → `volleyball_persons_best.pt`
- `BallTrain_VF.ipynb` — trains the ball detector → `volleyball_ball_best.pt`
- `TrainLSTM_temporal_Final.ipynb` — trains the trajectory model → `best_lstm_tracker.pth`
- `v11_final_EN.ipynb` — the integrated application: loads every model, trains the CNN and the team MLP, and launches the GUI.

5. How to Run

The whole system is launched from the `v11_final` notebook. The cells are grouped into numbered sections and must be executed in order:

1. Section 0 — install dependencies. The Gemma 4 / Unsloth install takes two to three minutes.
2. Sections 1–8 — run the definition cells (constants, network architectures, homography and minimap, detection function, ball tracker, colour features, Gemma loader, pipeline class).
3. Section 9 — upload the weight files, then run the cell to load every model and build the pipeline.
4. Section 10 — apply the pipeline patches (referee zones and minimap fix).
5. Sections 11–12 — the team MLP (see the weights logic below).
6. Section 13 — run the single-frame test to confirm everything works.
7. Section 14 — launch the Gradio interface and use the application.

Weights: upload or train

Each model can either be reused from a pretrained weight file or trained from scratch:

Player and ball detectors: trained only in their own notebooks (`TrainPersonVF` and `BallTrain_VF`). Upload `volleyball_persons_best.pt` and `volleyball_ball_best.pt` to the application; they cannot be trained inside it.

Trajectory LSTM: trained only in TrainLSTM_temporal_Final. Upload best_lstm_tracker.pth; the application only loads it.

Role CNN: if best_cnn_classifier.pth is uploaded it is loaded and used; if it is missing the role would be random, but the application can train it from the “Train CNN” tab (this needs the volley_detection-5 dataset present in /content).

Team MLP: upload best_team_mlp.pt to reuse it; otherwise sections 11–12 train it from the match video using K-means labels. A simple file-existence check can be added at the start of those sections so it is not retrained when the weight is already present.

Gemma 4: downloaded automatically from Hugging Face on the first run, so no upload is needed. Set USE_GEMMA = False to skip it if VRAM is limited.

6. Design Decisions and Justification

The most important decision was to build our own ball dataset. We first tried the pretrained COCO ball class and it simply did not work on our video, so the extra effort of extracting and labelling frames was justified by the jump in detection quality. For the players we did the opposite and reused a ready-made Roboflow dataset, because team-labelled players were already available and there was no reason to re-annotate them.

Training the LSTM and the team MLP on our own match video was a deliberate domain choice: the camera angle, court and lighting at inference time are exactly the ones the models saw during training. The displacement target and the temporal split for the LSTM were chosen specifically so the reported numbers are honest; an absolute-position target would have produced a much lower MSE that mostly reflects “repeat the last position” rather than real prediction. For the team assignment we combine geometry and colour: a homography places each player on one side of the net, and the colour MLP overrides that decision when it is confident, which is more robust than either signal alone when the camera moves or players cross the net.

We used AI coding assistance during the project, mainly an LLM assistant (Claude). It was used to translate the notebook comments and interface to English, and to help structure this report. Every change was checked by running the affected part of the pipeline; we did not accept code we could not explain.

7. Conclusions and Lessons Learned

The system works end to end: from a raw clip it produces tracked players with team and role, a ball track, trajectory arrows, a minimap, and a language assistant, all from one interface. The part we are most satisfied with is the ball detector, because solving it required recognising that the public model was inadequate and building data ourselves.

The clearest limitation is the size of the validation sets, especially for the ball, where six images make the strict metrics unreliable. The LSTM's gain over the naive baseline is real but small, which is partly the nature of short-window volleyball motion and partly a consequence of using a single match for sequence data. If we did this again we would annotate more frames for both detectors, gather trajectory data from several matches, and consider fine-tuning the language model on volleyball captions instead of relying only on prompting.

Individual contributions

Angel worked on the vision and detection side: the player and ball YOLO detectors, the dataset preparation and visual metrics, the role-classification CNN, and the colour-based team MLP with its K-means labelling. Carlos worked on the sequence and tracking side: the ByteTrack integration, the trajectory extraction, and the design, training and evaluation of the LSTM, including the displacement target and the temporal-split methodology. Javier and Jorge worked on integration, interface and language: the Gradio GUI, the Gemma 4 assistant and its context prompting, the homography, minimap and camera-motion correction, and the full assembly of the pipeline together with the repository and documentation. The architecture decisions, the choice of sport and datasets, and the cross-checking of the final system were taken jointly by the three of us. All member helps in all parts of the Project.